Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 12:

- Learning a new language
- Introduction to C



A CSR Initiative by



NPTEL

Here We Go Again: Why Is It Difficult for Developers to Learn Another Programming Language?

By Nischal Shrestha, Colton Botta, Titus Barik, and Chris Parnin

DOI: 10.1145/3511062

- **Finding**: 61% of cross-language posts on Stack Overflow had **incorrect assumptions**

Interference with previous language knowledge

Language pair[1]	Posts[2]	% Accepted[3]	*n*	Correct[4] %
<C, C++>	30863	65%	9	30%
<C#, Visual Basic>	11522	62%	8	27%
<Objective-C, Swift>	9416	50%	10	33%
<Python, C++>	6763	51%	15	50%
<Java, C#>	6748	59%	16	53%
<Scala, Java>	6622	55%	8	27%
<PHP, Java>	6152	46%	16	53%
<R, Python>	2824	49%	12	40%
<Kotlin, Java>	2565	53%	6	20%
<Matlab, Python>	2407	53%	11	37%
<Node, PHP>	2077	40%	14	47%
<Ruby, Python>	1314	65%	14	47%
<Perl, Python>	1152	67%	13	43%
<Lua, C++>	1143	63%	12	40%
<Clojure, Java>	1098	68%	10	33%

R → Python

"I'm seriously confused. Maybe I'm thinking too much in **R** terms and can't wrap my head around what's going on in **Python**."

I am an intermediate level Python programmer and I want to learn C. What are the key similarities and key differences between the two languages that I should be most aware of?

Great choice! Moving from Python to C will expose you to a lot of new concepts, especially around low-level memory management, performance optimization, and system-level programming. Here's a breakdown of the key **similarities** and **differences** between Python and C to help you transition smoothly.

- What are the key differences?
  - Python is also **object-oriented** and **functional**
  - Python is **dynamically typed** and **interpreted** whereas C is **statically typed** and **compiled**
  - Key differences in datatypes (`int`, `float`, ...), function calling and return, handling memory

NPTEL

Python to C: Example 1

```
viraj@VIRAJ-KIAC:~$ python3 lec12.py
id(x) = 140193361640976
id(x) = 140193362571360
Positive
viraj@VIRAJ-KIAC:~$ echo $?
0
```

Run the python3 **interpreter** on the file `lec12.py`

labels an
nt object
ed

by **f-strings**

```
5       print(f'id(x) = {id(x)}')
6       if x > 0:
7           print("Positive")
8       elif x < 0:
9           print("Negative")
10      else:
11          print("Zero")
```

Many differences!

// (in C) is like # (in Python)
in C: No floor-division operator

Run the gcc **compiler** to convert `lec12.c` into an **executable program**
By default: `a.out` (or `a.exe`)

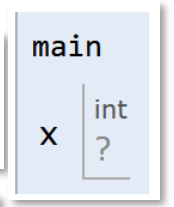
```
viraj@VIRAJ-KIAC:~$ gcc lec12.c
lec12.c: In function ‘main’:
lec12.c:4:20: warning: format ‘%d’ expects argument of type ‘int’,
 but argument 2 has type ‘int *’ [-Wformat=]
    4 |    printf("id(x) = %d\n", &x);
      |                    ~^     ~~
      |                     |     |
      |                     int   int *
      |                    %ls
lec12.c:6:20: warning: format ‘%d’ expects argument of type ‘int’,
 but argument 2 has type ‘int *’ [-Wformat=]
    6 |    printf("id(x) = %d\n", &x);
      |                    ~^     ~~
      |                     |     |
      |                     int   int *
      |                    %ls
viraj@VIRAJ-KIAC:~$ ./a.out
id(x) = -1629015564
id(x) = -1629015564
Negative
viraj@VIRAJ-KIAC:~$ echo $?
0
```

Compiler warnings (not **errors**):
This code may lead to unintended behaviour

Run the executable code `a.out` (in the current directory)

programming style

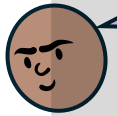
```
17  }
```



Python to C: Example 2

- PythonTutor (non-default visualization)

```
1  def inverse(n: int) -> float:
2      if n = 0:
3          return float('inf')
4      return 1 / n
5
6  if __name__ == '__main__':
7      n = 1001
8      print(f'inverse({n}) = {inverse(n)}')
```




```
viraj@VIRAJ-KIAC:~$ python3 inverse.py
  File "/home/viraj/inverse.py", line 2
    if n = 0:
       ^^^^^
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

Example 2: Attempt 1

```c
#include <stdio.h>
#include <math.h>

float inverse(int n) {
  if (n = 0) {
    return INFINITY;  // defined in math.h
  }
  return 1 / n;
}
```

```
Compiler warnings:
gcc-9 -std=gnu17 -Wall -Wno-unused-variable -Wno-unused-but-set-variable -Wno-main

 In function 'inverse':
5:7: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    5 |   if (n = 0) {
      |       ^
```

```c
  return 0;
}
```

No syntax error!

```
viraj@VIRAJ-KIAC:~$ gcc inverse.c
viraj@VIRAJ-KIAC:~$
```

```python
def inverse(n: int) -> float:
    if n = 0:
        return float('inf')
    return 1 / n

if __name__ == '__main__':
    n = 1001
    print(f'inverse({n}) = {inverse(n)}')
```

SyntaxError

`n = 0` is an

- assignment **statement** (Python)
- assignment **expression** (C)

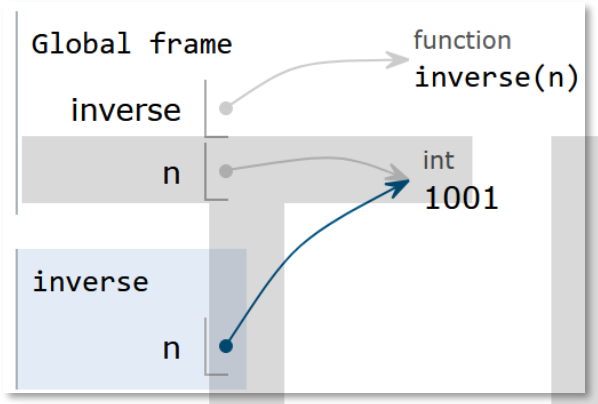
Example 2: Attempt 2

```
inverse(1001) = 0.00099900099000999
```

18 digits after the decimal

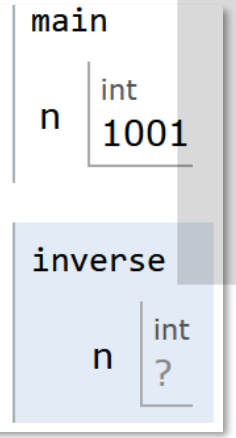
```
1  def inverse(n: int) -> float:
2      if n == 0:
3          return float('inf')
4      return 1 / n
```

Python: Pass **by assignment**




```
4  float inverse(int n) {
5    if (n == 0) {
6      return INFINITY;  // defined in math.h
7    }
8    return 1 / n;
9  }
10
```

PythonTutor



C: Pass **by value**

```
inverse(1001) = 0.00000000000000000
```

Example 2: Attempt 3

```c
#include <stdio.h>
#include <math.h>

double inverse(int n) {
  if (n == 0) {
    return INFINITY;  // defined in math.h
  }
  return 1.0 / n;
}

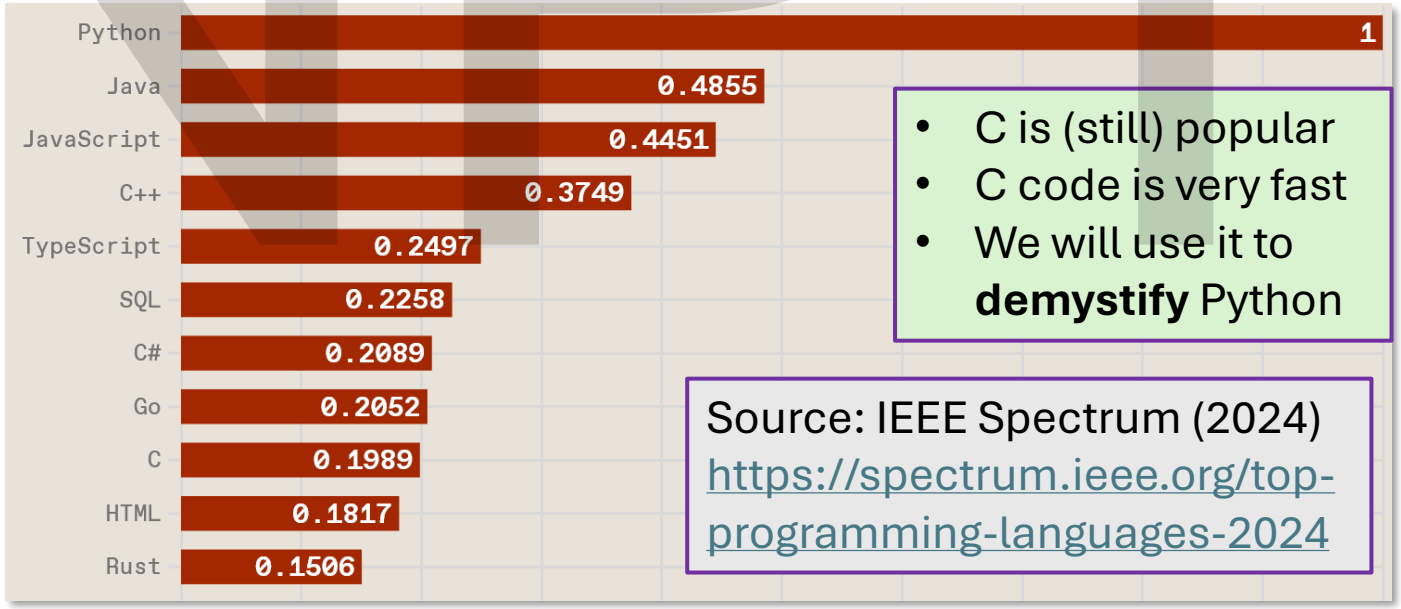
int main() {
  int n = 1001;
  printf("inverse(%d) = %.18f\n", n, inverse(n));
  return 0;
}
```

NPTEL

Summary

- Learning a new language can be tricky: some similarities, but **key differences**
- C (developed in the 1970's) is much older than Python (developed in the 1990's)

Gen AI tools can help (**technical prompting**)



- C is (still) popular
- C code is very fast
- We will use it to **demystify** Python

Source: IEEE Spectrum (2024) https://spectrum.ieee.org/top-programming-languages-2024

Interpreted (Python) vs. Compiled (

- Compare this Python code with this C code:
  - Error at run-time (Python) vs. **compile-time** (C)

```python
if __name__ == '__main__':
    s = 'Hi'
    x = 1.5
    print(s + x)
```

Dynamic typing:
this *might* be legal!

```c
#include <stdio.h>

int main() {
  char s[] = "Hi";
  float x = 1.5;
  printf(s + x);

  return 0;
}
```

Static typing:
this *is* illegal!

```
Done running (3 steps)
TypeError: can only concatenate str (not "float") to str
```



Is **`#include`** like **`import`**?
What is **`int main`**?
Why **`return 0`**?
Are **`{ }`** and **`;`** needed?
Is **`s[]`** a **`list`** or a **`str`**?
Is **`float`** the same?
How about **`int`**?
`printf(…)` ≈ **`print(f'…')`**?

Differences with `int`: Mutability, Limits

```
1  i = 1234567890
2  print(id(i))
3  i *= 3
4  print(id(i))
```

```
140371537191888
140371362397520
```

i	3703703670

```
4    int i = 1234567890;
5    printf("%p\n", &i);
6    i *= 3;
7    printf("%p\n", &i);
```

```
0xfff000bdc
0xfff000bdc
```

i	int -591263626

- Compare this Python code with this C code:
- An `int` in C is mutable and has finite precision (often: 32 bits)

No floor division in C:

- `//` ≈ Python's # comment
- `int / int` is integer division (it ***always*** truncates; ***not*** floor division)

User-defined functions

- Compare this Python function to this C function:

```python
def next_int(n: int) -> int:
    if n < 0:
        return n - 1
    return n + 1
```

```c
int next_int(int n) {
  if (n < 0) {
    return n - 1;
  }
  return n + 1;
}
```

Necessary

Not necessary in this case, but still useful!

- In C:
  - Fixed return type (can be `void`)
  - Each parameter h… type hint!)
  - Arguments are pa… int's n is a *copy of* caller's n)
  - What if the argument is large (e.g., a long string?)

Do not write:

```c
void main() { ... }
```